

Trabajo Practico Obligatorio (TPO)

Análisis, Diseño e Implementación de Software aplicando Patrones de Diseño

Objetivo

Diseñar, documentar e implementar en Java un sistema de software que resuelva una problemática determinada, aplicando adecuadamente patrones de diseño, principios SOLID y patrones GRASP,

Modalidad de trabajo

El trabajo se realizará en grupos de entre **2 a 4 integrantes**.

Cada grupo deberá analizar, diseñar una solución orientada a objetos e implementarla en Java.

El proyecto deberá ser gestionado mediante un repositorio Git, preferentemente en GitHub, donde pueda observarse la participación individual de cada integrante.

Problemáticas sugeridas

En archivo Anexo_PropuestaTPO

Requisitos generales del sistema

El sistema desarrollado deberá cumplir con los siguientes requisitos mínimos:

- Estar implementado en Java.
- Utilizar programación orientada a objetos.
- Contar con una estructura clara de paquetes.
- Incluir clases de dominio, servicios/controladores y clases de prueba o ejecución.
- Aplicar al menos:
 - patrón creacional.
 - patrón estructural.
 - patrones de comportamiento.
 - 3 principios SOLID.
 - 3 patrones GRASP.
- Presentar diagramas UML coherentes con la solución implementada.
- Justificar por escrito las decisiones de diseño adoptadas.
- Permitir ejecutar una demostración funcional del sistema.

Entrega en tres fases

Fase 1: Análisis del problema y diseño inicial (3 de Junio)

Objetivo

Comprender la problemática, definir el alcance del sistema y construir un primer modelo de análisis.

Entregables

Cada grupo deberá presentar:

1. Descripción general de la problemática.
2. Objetivos del sistema.
3. Alcance funcional.
4. Requisitos funcionales.
5. Requisitos no funcionales básicos.
6. Diagrama de Clases
7. Identificación de actores o usuarios.
8. Casos de uso principales.
9. Diagrama de casos de uso.
10. Primer listado de clases candidatas.
11. Repositorio Git creado y compartido con la docente.

Se evaluará

- Claridad en la definición del problema.
- Coherencia en los diagramas presentados
- Correcta delimitación del alcance.
- Participación inicial de todos los integrantes en el repositorio.
- Organización del grupo y distribución preliminar de tareas.

Fase 2: Diseño orientado a objetos y patrones (10 de Junio)

Objetivo

Diseñar la solución orientada a objetos, definir responsabilidades y seleccionar los patrones de diseño que serán aplicados.

Entregables

Cada grupo deberá presentar:

1. Diagrama de clases UML.
2. Diagrama de secuencia de al menos 2 casos de uso relevantes.
3. Identificación de patrones de diseño a aplicar.
4. Justificación de cada patrón seleccionado.
5. Identificación de principios SOLID aplicados.
6. Identificación de patrones GRASP aplicados.
7. Estructura inicial de paquetes del proyecto Java.
8. Primera versión del código con clases principales, interfaces y relaciones.
9. Informe breve de avance individual por integrante.

Se evaluará

- Coherencia del diagrama de clases.
 - Correcta asignación de responsabilidades.
 - Uso pertinente de patrones.
 - Justificación conceptual de SOLID y GRASP.
 - Correspondencia entre UML y código.
 - Evidencia de avances individuales en Git.
-

Fase 3: Implementación y prueba (20 de Junio)

Objetivo

Finalizar la implementación del sistema, validar su funcionamiento y defender las decisiones de diseño adoptadas.

Entregables

Cada grupo deberá presentar:

1. Proyecto Java completo y ejecutable.
2. Código organizado en paquetes.
3. Implementación funcional de los casos de uso principales.
4. Aplicación concreta de los patrones seleccionados.
5. Casos de prueba o escenarios de ejecución.
6. Informe final del proyecto.
7. Diagramas UML actualizados.
8. Registro de commits individuales.

Se evaluará

- Funcionamiento del sistema.
- Calidad del código.
- Coherencia entre análisis, diseño e implementación.
- Correcta aplicación de patrones.
- Capacidad de justificar decisiones técnicas.
- Participación individual comprobable.

Seguimiento grupal e individual

Para garantizar el seguimiento del trabajo, cada grupo deberá utilizar Git desde el inicio del proyecto.

Requisitos mínimos del repositorio

El repositorio deberá contener:

- Código fuente del proyecto.

- Carpeta /documentacion.
- Diagramas UML.
- Informe de cada fase.
- Archivo README .md.
- Registro de tareas por integrante.
- Historial de commits significativo.

El archivo README deberá incluir

- Nombre del proyecto.
 - Integrantes del grupo.
 - Descripción breve del sistema.
 - Instrucciones para ejecutar el proyecto.
 - Patrones aplicados.
 - Principios SOLID aplicados.
 - Patrones GRASP aplicados.
 - Distribución de tareas.
-

Seguimiento individual

Cada integrante deberá demostrar participación real en el proyecto.

Se tendrá en cuenta:

- Cantidad y calidad de commits.
- Tareas asignadas y cumplidas.
- Participación en el diseño.
- Participación en la codificación.
- Explicación individual durante la defensa.
- Conocimiento del código desarrollado.
- Capacidad para responder preguntas sobre patrones, clases y responsabilidades.

No se evaluará únicamente la cantidad de commits, sino su relevancia. Un commit significativo debe representar un avance concreto: implementación de una clase, corrección de errores, incorporación de pruebas, mejora del diseño, documentación o refactorización.

Informe final

El informe final deberá contener:

1. Carátula.
2. Introducción.
3. Descripción del problema.
4. Objetivos del sistema.
5. Requisitos funcionales.
6. Casos de uso.
7. Diagrama de clases.

8. Diagramas de secuencia.
9. Descripción de la arquitectura del sistema.
10. Patrones de diseño aplicados.
11. Justificación de SOLID.
12. Justificación de GRASP.
13. Explicación de las clases principales.
14. Capturas de ejecución o ejemplos de uso.
15. Conclusión grupal.
16. Reflexión individual de cada integrante.

Defensa oral (segundo parcial - 24 de Junio)

Durante la defensa final, cada grupo deberá presentar:

- La problemática abordada.
- El diseño general de la solución.
- Los patrones aplicados.
- La relación entre UML y código.
- Una demostración del sistema funcionando.
- Las principales dificultades encontradas.
- Las mejoras posibles.

Además, cada integrante deberá responder preguntas individuales sobre el código y las decisiones de diseño.

Observaciones importantes para los estudiantes

Cada patrón debe responder a una necesidad concreta del diseño.

Se valorará especialmente que el grupo pueda explicar por qué decidió utilizar un patrón determinado, qué problema resuelve dentro del sistema y qué ventajas aporta frente a una solución más rígida o acoplada.

También se tendrá en cuenta la evolución del proyecto. Por eso, las tres fases no deben ser entregas aisladas, sino etapas progresivas de un mismo desarrollo.

El producto final deberá mostrar coherencia entre:

problema → análisis → diseño → patrones → código → pruebas → defensa.